**PYTHON PROGRAMMING**

**NAME:M.Sharmila**

**Reg no: 192572139**

**GITHUB: https://github.com/sharmilamohan12/python/tree/main/CO%20TOOLS**

**ASSIGNMENT 3 – CO3**

**CO3 Coverage:** Implement Python data structures such as lists, tuples, dictionaries and sets to store, access and manipulate unstructured data effectively – operations, slicing, mutability, copying and list comprehension.

**Nature of Assignment:** Individual
**Duration:** 1 Week
**Total Marks:** 50

**Q1 – Lists and Slicing: Daily Rainfall Records**

**6 Marks – K3 (Apply)**

Below is the complete assignment content, including Python implementation, pseudocode, complexity analysis and experimental investigation.

**1. Problem Understanding**

A village rain-gauge station records rainfall in millimetres (mm) every day for 365 days.

The rainfall readings are stored in a Python list. The program analyses the data to calculate:

1. Weekly rainfall totals.
2. Wettest and driest weeks.
3. Maximum and minimum rainfall readings with their positions.
4. Seven-day moving average.
5. List of dry days (0 mm).

```
print(readings)


# 1. Calculate Weekly Rainfall Totals
weekly_totals = []

for i in range(0, len(readings), 7):
    week = readings[i:i+7]
    total = sum(week)
    weekly_totals.append(total)

print("\nWeekly Rainfall Totals:")
print(weekly_totals)


# 2. Find Maximum and Minimum Rainfall
maximum = max(readings)
minimum = min(readings)

max_position = readings.index(maximum)
min_position = readings.index(minimum)

print("\nMaximum Rainfall:", maximum, "mm")
print("Maximum Position: Day", max_position + 1)

print("\nMinimum Rainfall:", minimum, "mm")
print("Minimum Position: Day", min_position + 1)


# 3. Find Wettest and Driest Weeks
wettest_week = weekly_totals.index(max(weekly_totals))
driest_week = weekly_totals.index(min(weekly_totals))

print("\nWettest Week:", wettest_week + 1)
print("Wettest Week Total:", max(weekly_totals), "mm")

print("\nDriest Week:", driest_week + 1)
print("Driest Week Total:", min(weekly_totals), "mm")


# 4. Calculate 7-Day Moving Average
moving_average = []

for i in range(len(readings) - 6):
    total = sum(readings[i:i+7])
    average = total / 7
    moving_average.append(average)
```

Pseudocode for 7-Day Moving Average

START

INPUT rainfall readings

CREATE empty moving_average list

FOR i = 0 TO length(readings) - 7

SET total = 0

FOR j = i TO i + 6

total = total + readings[j]

END FOR

average = total / 7

ADD average to moving_average

END FOR

DISPLAY moving_average

STOP

Alternative Concept and Complexity Analysis

```
from collections import deque

readings = [10, 20, 30, 40, 50, 60, 70, 80, 90]

window = deque(maxlen=7)

for value in readings:

    window.append(value)

    if len(window) == 7:

        average = sum(window) / 7

        print("7-Day Average:", average)
```

Output:

```
Dry Days:
[27, 96, 136, 331]

==================================
7-Day Average: 40.0
7-Day Average: 50.0
7-Day Average: 60.0

==================================
7-Day Average: 40.0
7-Day Average: 50.0
7-Day Average: 60.0

==================================
7-Day Average: 40.0
7-Day Average: 50.0
7-Day Average: 60.0
```

**Q2 – Tuples, Mutability and Copying**

**7 Marks – K4 (Analyse)**

**1. Problem Understanding**

A city bus service stores GPS coordinates of bus stops using latitude and longitude.

Each bus stop is represented using a tuple containing two values:

IMPLEMENTATION:

```
rows, cols = 3, 4

# Incorrect grid
bad = [[0] * cols] * rows

bad[0][1] = 7

# TODO 1: Correct grid using list comprehension
good = [[0 for j in range(cols)] for i in range(rows)]

good[0][1] = 7

print("bad :", bad)
print("good:", good)

print(bad[0] is bad[1], good[0] is good[1])

# Tuples as dictionary keys
stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar"
}

print(stops[(13.0418, 80.2341)])
```

OUTPUT:

```
>>>
============================================================
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar
>>>
```

Representing Bus Stops Using Tuples

```
stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar",
    (13.0674, 80.2376): "Nungambakkam"
}

# Access bus stop
print(stops[(13.0418, 80.2341)])

# Unpacking tuples
for (latitude, longitude), name in stops.items():

    print("Bus Stop:", name)
    print("Latitude:", latitude)
    print("Longitude:", longitude)
```

OUTPUT:

```
>>>
==================================
T. Nagar
Bus Stop: Chennai Central
Latitude: 13.0827
Longitude: 80.2707
Bus Stop: T. Nagar
Latitude: 13.0418
Longitude: 80.2341
Bus Stop: Nungambakkam
Latitude: 13.0674
Longitude: 80.2376
>>>
```

Shallow Copy – copy.copy():

```
a = [10, 20, 30, 40]

b = a[:]

b[0] = 100

print("Original:", a)
print("Copied:", b)
```

OUTPUT:

```
>>>
==================================
Original: [10, 20, 30, 40]
Copied: [100, 20, 30, 40]
>>>
```

**Q3 – Dictionaries: Farmer Helpline Complaint Analysis**

Below is the complete assignment answer with Python programs, explanations, experimental investigation, expected outputs and complexity analysis.

**1. Problem Understanding**

The Kisan Call Centre receives complaints from farmers about seeds, fertilisers, water shortages, crop diseases and subsidies.

IMPLEMENTATION:

```
words = [
    "seeds", "price", "seeds",
    "water", "price", "seeds"
]

distinct = set(words)

counts = {}

for word in distinct:

    counts[word] = words.count(word)

print(counts)
```

OUTPUT:

```
==========================================
{'water': 1, 'seeds': 3, 'price': 2}
>>>
```

```
for n in sizes:

    complaints = generate_complaints(n)

    # Dictionary timing
    start = time.perf_counter()

    d = dictionary_count(complaints)

    dictionary_time = time.perf_counter() - start

    # List count timing
    start = time.perf_counter()

    l = list_count(complaints)

    list_time = time.perf_counter() - start

    # Counter timing
    start = time.perf_counter()

    c = counter_count(complaints)

    counter_time = time.perf_counter() - start

    print("\nNumber of Complaints:", n)

    print("Dictionary Time: {:.6f} seconds".format(dictionary_time))

    print("List.count Time: {:.6f} seconds".format(list_time))

    print("Counter Time: {:.6f} seconds".format(counter_time))

    print("Results Match:", d == l == c)


# Edge Case Testing
print("\nEDGE CASE TESTING")

print("Empty Text:", dictionary_count([""]))

print("Uppercase and Lowercase:")

print(dictionary_count([
    "SEEDS seeds Seeds"
]))
```

**Experimental Investigation**

The experiment compares three approaches:

1. Dictionary method.
2. list.count() method.
3. collections.Counter method.

It generates 1,000, 10,000 and 100,000 complaints and measures their execution times.

**Using collections.Counter:**

**IMPLEMENTATION:**

```
from collections import Counter

words = [
    "seeds", "price", "seeds",
    "water", "price", "seeds"
]

counts = Counter(words)

print(counts)

print(counts.most_common(2))
```

**OUTPUT:**

```
============================================================
Counter({'seeds': 3, 'price': 2, 'water': 1})
[('seeds', 3), ('price', 2)]
>
```

**Q4 – Sets: Elective Registration Clash Detection (7 Marks)**

**1. Complete the Code Using Set Operators**

**Completed Python program:**

```python
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}

all_students = {f"1923110{i:02d}" for i in range(1, 11)}

# TODO 1: Using set operators

# Students registered in both electives
both = ai_elective & iot_elective

# Students registered in exactly one elective
exactly_one = ai_elective ^ iot_elective

# Students who registered in neither elective
neither = all_students - (ai_elective | iot_elective)

print("clash      :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither    :", sorted(neither))
```

**Output:**

```text
clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']
```

OUTPUT:

```
========== RESTART:
{'water': 1, 'seeds': 3, 'price': 2}
>>>
========== RESTART:
Counter({'seeds': 3, 'price': 2, 'water': 1})
[('seeds', 3), ('price', 2)]
>>>
```

Test Empty and Identical Electives

```python
# Empty electives
ai = set()
iot = set()

print("Empty intersection:", ai & iot)
print("Empty union:", ai | iot)

# Identical electives
ai = {"101", "102", "103"}
iot = {"101", "102", "103"}

print("Identical intersection:", ai & iot)
print("Identical difference:", ai ^ iot)
```

OUTPUT:

```
==========
Empty intersection: set()
Empty union: set()
Identical intersection: {'101', '102', '103'}
Identical difference: set()
>>>
```

**Q5 – List Comprehension: Cleaning Sensor Readings**

**Complete Python Code**

This program performs all three methods:

1. For loop with append()
2. List comprehension
3. map() and filter() with lambda

It also converts Celsius to Fahrenheit, identifies alarm minutes, compares execution time and measures memory usage.

```
# --------------------------------
# VERIFY ALL THREE METHODS
# --------------------------------

print("\nVerification:")

print("Celsius results are same:",
      valid1 == valid2 == valid3)

print("Fahrenheit results are same:",
      fahrenheit1 == fahrenheit2 == fahrenheit3)


# --------------------------------
# MEMORY COMPARISON
# --------------------------------

list_result = [c for c in readings if -50 <= c <= 50]

generator_result = (
    c for c in readings if -50 <= c <= 50
)

print("\nMemory Usage:")
print("List:", sys.getsizeof(list_result), "bytes")
print("Generator:", sys.getsizeof(generator_result), "bytes")
```

### Expected Output

```text
Original Readings: [2, 5, 9, 12, -55, 4, 7, 52, 6, -10]

Valid Celsius Readings: [2, 5, 9, 12, 4, 7, 6, -10]

Fahrenheit Readings: [35.6, 41.0, 48.2, 53.6, 39.2, 44.6, 42.8, 14.0]

Alarm Minutes: [3, 4, 5, 8, 10]

Verification:
Celsius results are same: True
```

6. **Q6 – Nested Data: A Railway Timetable (7 Marks)**

**1. Complete Python Program**

The following program uses a nested dictionary containing 5 trains. It finds trains stopping at a given station on a particular day, sorts them by departure time, and prints the timetable.

IMPLEMENTATION:

```
File  Edit  Format  Run  Options  Window  Help
```

```
for train_no, details in trains.items():

    if day in details["days"]:

        for stop in details["stops"]:

            if stop[0].lower() == station.lower():

                result.append((
                    train_no,
                    details["name"],
                    stop[1],
                    stop[2]
                ))

# Sort trains by departure time
result = sorted(result, key=lambda x: x[3])

# Display results
print("\nTrains stopping at", station, "on", day)

if result:
    print("\nTrain No\tTrain Name\t\tArrival\tDeparture")

    for train in result:
        print(train[0], "\t", train[1], "\t", train[2], "\t", train[3])

else:
    print("No trains found.")


# Print complete timetable
print("\n========== COMPLETE RAILWAY TIMETABLE ==========")

for train_no, details in trains.items():

    print("\nTrain Number:", train_no)
    print("Train Name:", details["name"])
    print("Running Days:", sorted(details["days"]))

    print("Station\t\tArrival\tDeparture")

    for stop in details["stops"]:
        print(stop[0], "\t", stop[1], "\t", stop[2])
```

OUTPUT:

```
Empty intersection: set()
Empty union: set()
Identical intersection: {'101', '102', '103'}
Identical difference: set()
>>>
```

**2. Nested Data Structure**

The timetable uses the following structure:

Trains (Dictionary)

|

|-- 12631

|  |

|  |-- name: Nellai Express

|  |

|  |-- stops: List

|  |  |

|  |  |-- (Chennai, 20:10, 20:20)

|  |  |-- (Villupuram, 22:00, 22:05)

|  |  |-- (Trichy, 02:30, 02:35)

|  |

|  |-- days: Set

|  |

|  |-- Monday

|  |-- Wednesday

|

|-- 12632

|-- 12633

|-- 12634

|-- 12635

Q7 – Design a Contact-Merging Program from First Principles (

IMPLEMENTATION:

```python
def normalize_phone(phone):

    if not phone:
        return ""

    digits = ""

    for ch in phone:
        if ch.isdigit():
            digits += ch

    if len(digits) == 12 and digits.startswith("91"):
        digits = digits[2:]

    return digits
```

Normalization of Names:

```python
def normalize_name(name):

    name = name.lower().strip()

    if name.startswith("dr."):
        name = name[3:].strip()

    return name
```

```
print("========== MERGED CONTACTS ==========")

for contact_id, details in result.items():

    print("\nContact ID:", contact_id)

    print("Names:", sorted(details["names"]))

    print("Phone Numbers:", sorted(details["phones"]))

    print("Email Addresses:", sorted(details["emails"]))


# ---------------------------------
# PERFORMANCE TEST
# ---------------------------------

print("\n========== PERFORMANCE TEST ==========")

for n in [1000, 100000]:

    test_contacts = []

    for i in range(n):

        test_contacts.append({
            "name": "Person " + str(i),
            "phones": ["900000" + str(i).zfill(4)],
            "emails": ["person" + str(i) + "@gmail.com"]
        })

    start = time.perf_counter()

    merged_result = merge_contacts(test_contacts)

    end = time.perf_counter()

    print("\nInput Contacts:", n)

    print("Merged Contacts:", len(merged_result))

    print("Execution Time:", round(end - start, 6), "seconds")
```

**Conclusion**

The contact-merging program uses dictionaries and sets to combine duplicate contacts efficiently.

- Phone numbers and email addresses are normalized before comparison.
- Dictionaries provide fast contact lookup.
- Sets remove duplicate phone numbers and email addresses.
- Missing information is handled without stopping the program.
- The dictionary-based approach is more efficient than comparing every pair of contacts